Library Management System - Step by Step

Spring Boot MVC + JPA + JSP + Hibernate + PostgreSQL | Add Book + View All Books (+ Search) | **Yellow = memorize**

A. Setup

Step 1. pgAdmin 4 > Servers > PostgreSQL (type password) > right-click **Databases** > Create > Database > name **library** > Save

- If it says *connection timeout*: Win+R > services.msc > postgresql-x64-XX > Start.

Step 2. STS > File > New > Spring Starter Project > Name **BookApp**, package **com.example.demo** > Next > tick **Spring Web**, **Spring Data JPA**, **PostgreSQL Driver** > Finish

Step 3. Open **pom.xml** > inside **<dependencies>** add this > Ctrl+S > right-click project > Maven > Update Project

```
<dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
</dependency>
```

- Story: Apache made Tomcat, embedded in Spring Boot; its JSP engine is Jasper. groupId = dots, artifactId = dashes.

Step 4. Open **src/main/resources/application.properties** > add below the first line > Ctrl+S

```
spring.application.name=BookApp

spring.datasource.url=jdbc:postgresql://localhost:5432/library
spring.datasource.username=postgres
spring.datasource.password=your_password
spring.jpa.hibernate.ddl-auto=update
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp
```

- URL has only COLONS: jdbc**:**postgresql**://**localhost**:**5432/library (never ;).
- Use the pgAdmin password of the lab machine.

B. Java classes (right-click com.example.demo > New)

Step 5. New > **Class** > **Book** > type this > **Ctrl+Shift+O** (choose jakarta) > click above last } > **Alt+Shift+S** > Generate Getters and Setters > Select All > Generate > Ctrl+S

```
package com.example.demo;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Book {
    @Id
    private int bookId;
    private String title;
    private String author;
    private String publisher;
    private String isbn;
    private String edition;
    private int publicationYear;
    private String category;
    private int availableCopies;
    public Book() {}

    // getters and setters (generated by STS)
}
```

- Numbers → int, text → String, names in camelCase (Available Copies → availableCopies).
- Check the tab: **Book.java** ● (dot) = not saved. Press Ctrl+S.

Step 6. New > **Interface** > **BookRepository** > Ctrl+Shift+O > Ctrl+S

```
package com.example.demo;

import org.springframework.data.jpa.repository.JpaRepository;

public interface BookRepository extends JpaRepository<Book, Integer> {
}
```

- <Entity, ID type>. Use Integer, not int. Free methods: save, findAll, findById, deleteById.

Step 7. New > Class > **BookService** > Ctrl+Shift+O (choose java.util.List) > Ctrl+S

```
package com.example.demo;

import java.util.List;
import org.springframework.stereotype.Service;

@Service
public class BookService {
    private final BookRepository repo;
    public BookService(BookRepository repo) {
        this.repo = repo;
    }
    public void save(Book b) {
        repo.save(b);
    }
    public List<Book> getAll() {
        return repo.findAll();
    }
    public Book getById(int id) {
        return repo.findById(id).orElse(null);
    }
}
```

- Forgot **@Service**? Error: *required a bean of type BookService that could not be found*.
- 3 labels to remember: **@Entity, @Service, @Controller**.

Step 8. New > Class > **BookController** > Ctrl+Shift+O (Model = org.springframework.ui.Model) > Ctrl+S

```
package com.example.demo;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
public class BookController {
    private final BookService service;
    public BookController(BookService service) {
        this.service = service;
    }

    @GetMapping("/")
    public String form() {
        return "book-form";
    }

    @PostMapping("/save")
    public String save(Book b) {
        service.save(b);
        return "book-form";
    }

    @GetMapping("/books")
    public String list(Model model) {
        model.addAttribute("books", service.getAll());
        return "book-list";
    }

    @GetMapping("/search")
    public String search(@RequestParam("id") int id, Model model) {
        model.addAttribute("book", service.getById(id));
        return "book-search";
    }
}
```

- Service and Controller have the SAME top: private final field + constructor.
- return "book-form" → opens /WEB-INF/views/book-form.jsp (prefix + name + suffix).
- Search method (grey) is optional - see Step 12.

C. JSP pages

Step 9. In the LOWER **src** folder: right-click **main** > New > Folder > type **webapp/WEB-INF/views** > Finish

- Not src/main/java. WEB-INF in capitals.

Step 10. Right-click **views** > New > File > **book-form.jsp** > Ctrl+S

```
<h2>Library Management System</h2>

<form action="/save" method="post">
  Book ID: <input name="bookId"><br><br>
  Title: <input name="title"><br><br>
  Author: <input name="author"><br><br>
  Publisher: <input name="publisher"><br><br>
  ISBN: <input name="isbn"><br><br>
  Edition: <input name="edition"><br><br>
  Year: <input name="publicationYear"><br><br>
  Category: <input name="category"><br><br>
  Copies: <input name="availableCopies"><br><br>
  <input type="submit" value="Add Book">
</form>

<br><a href="/books">View All Books</a>
```

- input **name** = entity field (exact spelling). action="/save" post = @PostMapping("/save").
- Link href = the URL in @GetMapping ("/books"), never the JSP file name.

Step 11. Right-click **views** > New > File > **book-list.jsp** > Ctrl+S

```
<%@ page import="java.util.*, com.example.demo.Book" %>
<h2>All Books</h2>

<% for (Book b : (List<Book>) request.getAttribute("books")) { %>
  Book Id: <%= b.getBookId() %><br>
  Title: <%= b.getTitle() %><br>
  Author: <%= b.getAuthor() %><br>
  <hr>
<% } %>

<br><a href="/">Add New Book</a>
```

- Only 2 Java lines: **<% for (...) { %>** opens the loop, **<% } %>** closes it.
- Every <% needs ONE %>. Never put <% inside another <%. List page has NO if/else.
- "books" in addAttribute = "books" in getAttribute (same word).

D. Run and check

Step R. Ctrl+Shift+S (save all) > right-click **BookApp** > Run As > **Spring Boot App** > console shows **Tomcat started on port 8080**

- Browser: **localhost:8080/** > fill all boxes > Add Book > click View All Books.
- pgAdmin: library > Query Tool > **SELECT * FROM book;** > F5 > your rows are there.
- Stop with the red square before running again. JSP changes: just Ctrl+S and refresh the browser.

E. Optional - Search by ID

Step 12. Add a search box to **book-list.jsp** (below <h2>), and create a new file **book-search.jsp**

```
<!-- in book-list.jsp -->
<form action="/search">
  Search ID: <input name="id"> <input type="submit" value="Search">
</form>

<!-- views/book-search.jsp -->
<%@ page import="com.example.demo.Book" %>
<h2>Search Result</h2>
<% Book b = (Book) request.getAttribute("book");
   if (b != null) { %>
  Book Id: <%= b.getBookId() %><br>
  Title: <%= b.getTitle() %><br>
<% } else { %>
  Book not found
<% } %>
<br><a href="/books">Back</a>
```

- List page = MANY → for loop. Search page = ONE → if / else.
- name="id" = @RequestParam("id").

F. Errors you met + fix

Error	Fix
pgAdmin: connection timeout expired	services.msc > postgresql > Start
Failed to determine a suitable driver class	typo in url (; instead of :) or properties not saved
Red X on Book.java, code looks right	file not saved (dot on tab) > Ctrl+S
bean of type BookService could not be found	add @Service above public class BookService
Whitelabel 500 on /books	wrong <% %> in JSP, or "books" names don't match
Whitelabel 404	JSP name/folder wrong, or link points to .jsp instead of URL
Whitelabel 400 on submit	a number box (ID, Year, Copies) left empty or has text

Memory chain: pgAdmin DB → STS (3 ticks) → pom jasper → properties → Entity → Repo → Service → Controller → folder → form JSP → list JSP → Run.